

Linux AppArmor & SSH MFA Security Lab

Application Confinement and
Nmap Scan Restrictions

Valentin Ochioiu
Network Security Student

Folkuniversitetet, Gothenburg
13 February 2026

Project Overview

This project explores how AppArmor confines Linux applications through custom profiles and audit-driven refinement. It also strengthens SSH access with multi-factor authentication and tests a targeted restriction on Nmap raw-socket scans.

Linux process confinement

- ▶ Inspect AppArmor status and profile modes.
- ▶ Create and refine profiles for cowsay and lnnav.
- ▶ Use application output and audit logs to diagnose policy restrictions.

Further Security Hardening

Configure SSH multi-factor authentication with a password and authenticator code. Restrict Nmap raw-socket scanning through an AppArmor profile.

Security objective

Preserve required application behaviour while limiting access and privileged operations.

AppArmor foundation

- ▶ Ubuntu VM with AppArmor enabled.
- ▶ Isolated Linux test environment.
- ▶ Applications: man, cowsay and lnav.
- ▶ Generate a profile, exercise the program, review logs and reload the policy.

Further Security Hardening

- ▶ SSH connection to the Linux test server.
- ▶ Password plus authenticator verification code.
- ▶ Nmap scans a designated test host.
- ▶ Compare an unprivileged scan with a privileged SYN scan.

AppArmor Status and Tools

```
ubuntu@ubuntu:~$ sudo aa-status
apparmor module is loaded.
164 profiles are loaded.
65 profiles are in enforce mode.
/snap/snapd/24792/usr/lib/snapd/snap-confine
/snap/snapd/24792/usr/lib/snapd/snap-confine//mount-namespace-capture-helper
/snap/snapd/25935/usr/lib/snapd/snap-confine
/snap/snapd/25935/usr/lib/snapd/snap-confine//mount-namespace-capture-helper
/usr/bin/evince
/usr/bin/evince-previewer
/usr/bin/evince-previewer//sanitized_helper
/usr/bin/evince-thumbnailer
/usr/bin/evince//sanitized_helper
/usr/bin/evince//snap_browsers
/usr/bin/man
```

Captured baseline: module loaded, 164 profiles loaded and 65 in enforce mode.

Baseline checks

- ▶ Confirm that the kernel module and profiles are loaded.
- ▶ Install apparmor-utils for profile management.
- ▶ Check each application profile rather than relying only on the total profile count.

Complain and Enforce Modes

```
ubuntu@ubuntu:~$ sudo aa-complain /etc/apparmor.d/usr.bin.man  
Setting /etc/apparmor.d/usr.bin.man to complain mode.
```

```
10 profiles are in complain mode.  
/usr/bin/man
```

Complain

Logs accesses that are missing an allowance, supporting profile development. Explicit deny rules can still block access.

Enforce

Applies policy restrictions. Retest normal application functions after changing the mode.

Cowsay: Profile Creation and Rule Review

```
ubuntu@ubuntu:~$ sudo aa-genprof cowsay
Enforce-mode changes:
Profiling: /usr/games/cowsay
Please start the application to be profiled in
another window and exercise its functionality now.
Once completed, select the "Scan" option below in
order to scan the system logs for AppArmor events.
For each AppArmor event, you will be given the
opportunity to choose whether the access should be
allowed or denied.
[(S)can system log for AppArmor events] / (F)inish
Reloaded AppArmor profiles in enforce mode.
Please consider contributing your new profile!
See the following wiki page for more information:
https://gitlab.com/apparmor/apparmor/wikis/Profiles
Finished generating profile for /usr/games/cowsay.
ubuntu@ubuntu:~$
```

Profile generation and a successful cowsay test.

```
ubuntu@ubuntu:~$ sudo aa-logprof
[sudo] password for ubuntu:
Updating AppArmor profiles in /etc/apparmor.d.
Reading log entries from /var/log/syslog.
Complain-mode changes:
Profile: /usr/games/cowsay
Path: /usr/share/cowsay/cows/default.cow
New Mode: r
Severity: unknown
[1 - include <abstractions/evince>]
2 - /usr/share/cowsay/cows/default.cow r,
(A)llow / [(D)eny] / [(I)gnore / (G)lob / Glob with (E)xtension / (N)ew / Audi(t) / Abo(r)t / (F)inish
Adding include <abstractions/evince> to profile.
Enforce-mode changes:
Profile: ubuntu_pro_esn_cache_systemd_detect_virt
Capability: perfmon
Severity: 7
```

aa-logprof identifies the template-file read request.

Profile refinement

The lab created a profile for /usr/games/cowsay, reviewed its template access and saved the updated policy. Review proposed rules against the function being tested.

lnav: Missing Profile and Recovery

```
ubuntu@ubuntu:~$ sudo aa-complain /usr/bin/lnav
Profile for /usr/bin/lnav not found, skipping
```

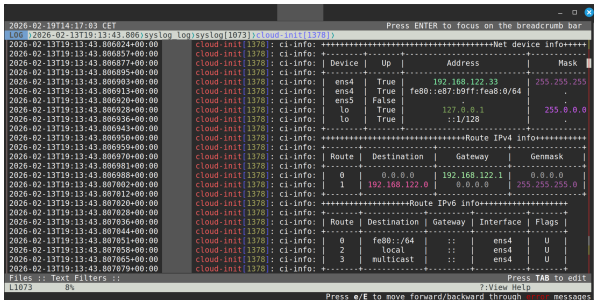
```
ubuntu@ubuntu:~$ sudo aa-genprof /usr/bin/lnav
Updating AppArmor profiles in /etc/apparmor.d.
Writing updated profile for /usr/bin/lnav.
Setting /usr/bin/lnav to complain mode.
```

```
ubuntu@ubuntu:~$ sudo apparmor_parser -r /etc/apparmor.d/usr.bin.lnav
[sudo] password for ubuntu:
```

Observed problem

- ▶ aa-complain reported that the lnav profile did not exist.
- ▶ aa-genprof created /usr/bin/lnav in complain mode.
- ▶ The lab exercised /var/log/syslog, saved permissions and reloaded the profile.

Inav: Functional Validation



```
2026-02-19T14:17:03 CET
LOG [2026-02-13T19:13:43.806]syslog Log[syslog[1073]]>cloud-init[1378]
2026-02-13T19:13:43.806024+00:00 cloud-init[1378]: ci-info: ++++++Net device info++++
2026-02-13T19:13:43.806057+00:00 cloud-init[1378]: ci-info: +-----+
2026-02-13T19:13:43.806077+00:00 cloud-init[1378]: ci-info: | Device | Up | Address | Mask |
2026-02-13T19:13:43.806095+00:00 cloud-init[1378]: ci-info: +-----+
2026-02-13T19:13:43.806093+00:00 cloud-init[1378]: ci-info: | ens4 | True | 192.168.122.33 | 255.255.255
2026-02-13T19:13:43.806913+00:00 cloud-init[1378]: ci-info: | ens4 | True | fe80::e87:b9ff:fea8:0/64 | .
2026-02-13T19:13:43.806920+00:00 cloud-init[1378]: ci-info: | ens5 | False | 127.0.0.1 | 255.0.0.0
2026-02-13T19:13:43.806928+00:00 cloud-init[1378]: ci-info: | lo | True | ::1/128 | .
2026-02-13T19:13:43.806936+00:00 cloud-init[1378]: ci-info: +-----+
2026-02-13T19:13:43.806943+00:00 cloud-init[1378]: ci-info: ++++++Route IPv4 info+++++
2026-02-13T19:13:43.806950+00:00 cloud-init[1378]: ci-info: +-----+
2026-02-13T19:13:43.806959+00:00 cloud-init[1378]: ci-info: | Route | Destination | Gateway | Genmask |
2026-02-13T19:13:43.806970+00:00 cloud-init[1378]: ci-info: +-----+
2026-02-13T19:13:43.806981+00:00 cloud-init[1378]: ci-info: | 0 | 0.0.0.0 | 192.168.122.1 | 0.0.0.0 |
2026-02-13T19:13:43.806988+00:00 cloud-init[1378]: ci-info: | 1 | 192.168.122.0 | 0.0.0.0 | 255.255.255.0 |
2026-02-13T19:13:43.807002+00:00 cloud-init[1378]: ci-info: +-----+
2026-02-13T19:13:43.807012+00:00 cloud-init[1378]: ci-info: ++++++Route IPv6 info+++++
2026-02-13T19:13:43.807020+00:00 cloud-init[1378]: ci-info: +-----+
2026-02-13T19:13:43.807028+00:00 cloud-init[1378]: ci-info: | Route | Destination | Gateway | Interface | Flags |
2026-02-13T19:13:43.807044+00:00 cloud-init[1378]: ci-info: +-----+
2026-02-13T19:13:43.807051+00:00 cloud-init[1378]: ci-info: | 0 | fe80::/64 | :: | ens4 | U |
2026-02-13T19:13:43.807058+00:00 cloud-init[1378]: ci-info: | 2 | local | :: | ens4 | U |
2026-02-13T19:13:43.807065+00:00 cloud-init[1378]: ci-info: | 3 | multicast | :: | ens4 | U |
2026-02-13T19:13:43.807079+00:00 cloud-init[1378]: ci-info: +-----+
Files :: Text Filters ::
L1073 8%
Press TAB to edit
?View Help
Press e/E to move forward/backward through messages
```

System-log view with lab addresses visible and host identifiers removed.

What this confirms

- ▶ Inav ran and displayed logs after profile refinement.
- ▶ Required access was reviewed with aa-logprof.
- ▶ A successful run in complain mode alone does not prove that an enforced profile permits every required operation.

SSH MFA Configuration

Further Security Hardening

/etc/pam.d/sshd – PAM stack

```
GNU nano 7.2 /etc/pam.d/sshd *
# PAM configuration for the Secure Shell service

# Standard Un*x authentication.
@include common-auth
auth required pam_google_authenticator.so
```

/etc/ssh/sshd_config – SSH daemon

```
# Change to yes to enable challenge-response passwords (beware issues with
# some PAM modules and threads)
KbdInteractiveAuthentication yes

# PAM authentication, then enable this but set PasswordAuthentication
# and KbdInteractiveAuthentication to 'no'.
UsePAM yes
```

Two configuration files

- ▶ The PAM stack includes normal UNIX authentication and requires the authenticator module.
- ▶ The SSH daemon configuration enables keyboard-interactive authentication and PAM integration.
- ▶ The two lower excerpts show SSH directives; they are not PAM stack rules.

Authenticator Enrollment

Further Security Hardening

WAYS TO SIGN IN OR VERIFY



One-time passwords enabled

You can use the one-time password codes generated by this app to verify your sign-ins

The authenticator account confirms that one-time passwords are enabled.

Enrollment recorded in the lab

- ▶ Run google-authenticator for the Ubuntu account.
- ▶ Register the account in the phone authenticator app.
- ▶ Rate limiting and prevention of code reuse were enabled.
- ▶ Settings and recovery material reside in the user account's `.google_authenticator` file.

Credential handling

For privacy and security, enrollment QR codes, shared secrets and recovery codes are confidential and are not disclosed in this presentation.

SSH MFA Validation

Further Security Hardening

```
[REDACTED]:~$ ssh ubuntu@192.168.122.47  
(ubuntu@192.168.122.47) Password:  
(ubuntu@192.168.122.47) Verification code: _
```

Visible evidence

The SSH client requests a password, followed by a verification code for the same Ubuntu account.

Reported outcome

The configured login required both factors. The capture ends at the code prompt and does not show the resulting authenticated shell.

Nmap Profile and Enforcement

Further Security Hardening

```
ubuntu@ubuntu:~$ sudo aa-genprof /usr/bin/nmap
Updating AppArmor profiles in /etc/apparmor.d.
Writing updated profile for /usr/bin/nmap.
Setting /usr/bin/nmap to complain mode.
```

```
ubuntu@ubuntu:~$ sudo nano /etc/apparmor.d/usr.bin.nmap

ubuntu@ubuntu:~$ sudo aa-enforce /usr/bin/nmap
Setting /usr/bin/nmap to enforce mode.
ubuntu@ubuntu:~$ sudo apparmor_parser -r /etc/apparmor.d/usr.bin.nmap
ubuntu@ubuntu:~$
```

Configuration sequence

Generate /usr/bin/nmap, edit /etc/apparmor.d/usr.bin.nmap, switch it to enforce mode and reload the profile with `apparmor_parser -r`.

Nmap File-Access Rules

Further Security Hardening

```
GNU nano 7.2 /etc/apparmor.d/usr.bin.nmap
abi <abi/3.0>,
include <tunables/global>

/usr/bin/nmap {
  include <abstractions/base>
  include <abstractions/consoles>
  include <abstractions/nameservice>

  /usr/share/nmap/** r,
  /usr/bin/nmap mr,
  /proc/*/net/dev r,
  owner /etc/nsswitch.conf r,
  owner /etc/passwd r,
  owner /usr/share/nmap/nmap-protocols r,
  owner /usr/share/nmap/nmap-services r,

  deny capability net_raw,
}
```

Required application resources

- ▶ /usr/share/nmap/** r,
Read Nmap data files.
- ▶ /usr/bin/nmap mr,
Permit read access and executable memory mapping.
- ▶ /proc/*/net/dev r,
Read network-interface information.
- ▶ The profile also includes base, consoles and nameservice abstractions.

Diagnosing an Unintended Nmap Failure

Further Security Hardening

```
ubuntu@ubuntu:~$ sudo nmap -sS 192.168.1.98
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-03-06 21:35 UTC
route_dst_netlink: can't find interface "ens5"
```

```
2026-03-06T21:35:55.978424+00:00 [redacted] kernel: audit: type=1400 audit(1772832955.976:3416): apparmor="DENIED" operation="open" class="file" profile="/usr/bin/nmap" name="/proc/1547/net/dev" pid=1547 comm="nmap" requested_mask="r" denied_mask="r" fsuid=0 ouid=0
2026-03-06T21:35:55.978467+00:00 [redacted] kernel: audit: type=1400 audit(1772832955.976:3417): apparmor="DENIED" operation="open" class="file" profile="/usr/bin/nmap" name="/proc/1547/net/dev" pid=1547 comm="nmap" requested_mask="r" denied_mask="r" fsuid=0 ouid=0
```

Observed failure

Nmap reports that it cannot find interface ens5.
The audit entry records denied read access to
/proc/1547/net/dev.

Lab correction

The profile needs /proc/*/net/dev r, to
read interface data. Restore necessary
file access while retaining the intended
raw-socket restriction.

Specific Nmap Restriction

Further Security Hardening

```
deny capability net_raw,
```

Policy intent

- ▶ Deny the net_raw capability to the confined Nmap process.
- ▶ Restrict scans that need raw sockets.
- ▶ Retain basic application functions supported by the remaining policy.

Demonstrated scope

The lab tests a privileged TCP SYN scan with -sS. It does not demonstrate a complete Nmap execution ban or show a separate UDP-scan test.

Allowed Scan and Blocked SYN Scan

Further Security Hardening

```
ubuntu@ubuntu:~$ nmap -Pn 192.168.1.98

Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-03-06 21:27 UTC
Nmap scan report for [REDACTED] (192.168.1.98)
Host is up (0.0012s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE
53/tcp    closed domain

Nmap done: 1 IP address (1 host up) scanned in 8.17 seconds
ubuntu@ubuntu:~$ sudo nmap -sS 192.168.1.98
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-03-06 21:28 UTC
Couldn't open a raw socket. Error: Permission denied (13)
ubuntu@ubuntu:~$ _
```

Why one worked

The unprivileged `nmap -Pn` scan uses normal TCP connections. It succeeds without raw sockets. `-Pn` skips host discovery.

Why one failed

The privileged `sudo nmap -sS` SYN scan needs raw sockets. AppArmor denies their creation, so the scan stops with permission denied.

Raw-Socket Denial in the Audit Log

Further Security Hardening

```
ubuntu@ubuntu:~$ sudo tail /var/log/syslog | grep raw
2026-03-06T21:13:27.816542+00:00 [REDACTED] kernel: audit: type=1400 audit(1772831607.814:3394): apparmor="DENIED" operation="create" class="net" info="failed type match" error=-13 profile="/usr/bin/nmap" pid=1447 comm="nmap" family="inet" sock_type="raw" protocol=255 requested="create" denied="create"
2026-03-06T21:20:00.492422+00:00 [REDACTED] kernel: audit: type=1400 audit(1772832000.489:3398): apparmor="DENIED" operation="create" class="net" info="failed type match" error=-13 profile="/usr/bin/nmap" pid=1465 comm="nmap" family="inet" sock_type="raw" protocol=255 requested="create" denied="create"
```

Fields visible in the capture

- ▶ apparmor="DENIED"
- ▶ profile="/usr/bin/nmap"
- ▶ operation="create"
- ▶ sock_type="raw"

Evidence interpretation

The log confirms AppArmor denied raw-socket creation. It reports a failed type match, not an explicit capability-denial event, so it does not isolate net_raw as the sole blocking rule.

Context beyond the demonstrated lab implementation

- ▶ Review profile abstractions and broad paths. A working application can still have more permission than it needs.
- ▶ Test each required workflow in enforce mode and keep a recovery path for restrictive policy changes.
- ▶ For SSH MFA, verify alternate authentication methods, invalid-code rejection and recovery access. The shown prompts do not prove coverage of every login path.
- ▶ Protect enrollment secrets and verify SSH host keys. Authenticator codes are not a substitute for server-identity verification.
- ▶ Nmap TCP connect scans can work without raw packets. Define the intended restriction before interpreting a successful scan as a policy failure.

Project Summary

Implemented

- ▶ AppArmor status and profile-mode checks.
- ▶ Custom cowsay and lnav profiles.
- ▶ SSH password and authenticator-code checks in the extended configuration.
- ▶ An enforced Nmap profile with a raw-capability restriction.

Evidence and lessons

- ▶ Profile generation, rule review and application output.
- ▶ SSH password and verification-code prompts.
- ▶ A completed ordinary scan and a blocked privileged SYN scan.
- ▶ Audit logs distinguish file-access problems from the intended scan restriction.

Security result

The lab combines stronger authentication with process confinement. Successful validation requires both permitted operations and the specific operation that the policy should deny.